FIELD GUIDE · FIRST-YEAR COMPUTER ENGINEERING

Your First Open Source Contribution

From zero to a merged pull request, and the roadmap to mastery



12
step contribution lifecycle

14
diagrams and mock-ups

6
phase mastery roadmap

14
day action plan

Assumes: fluency with Git and GitHub · ~30% of frontend web development · ~30% of system design

Edition: October 2026

Before you begin

This guide takes you from "I have never touched someone else's repository" to "I maintain part of one". It is written for first-year computer engineering students who already use Git and GitHub comfortably, and who are roughly 30% of the way through frontend web development and system design.

	What this guide assumes	What you do not need
Tools	Git (commit, branch, merge, remotes) and a GitHub account. A code editor and Node.js installed.	A powerful laptop, paid tools or a certificate.
Frontend (~30%)	HTML structure, CSS layout basics, JavaScript fundamentals, the DOM and events.	React, TypeScript or build-tool expertise. You will grow into them (Chapter 8).
System design (~30%)	Client-server model, HTTP requests and responses, what an API is, the idea of modules and layers.	Distributed-systems theory or scale experience.
Mindset	Patience, curiosity and willingness to ask questions in public.	Perfect English, a perfect GPA or prior internships.

How to use this guide

If you have...	Read	You will get
15 minutes	Chapter 1 and the lifecycle diagram in Chapter 5	The big picture and the motivation to start
One weekend	Chapters 4, 5 and 9	A project shortlist and a ready-to-run plan for your first pull request
The long haul	Chapter 8 (roadmap) and the checklists in Chapter 10	A path from first PR to maintainer, with checkpoints

> GOOD TO KNOW **About the examples**
> The project **pixel-ui**, the user names and the issue and PR numbers used in the screenshots and commands are **fictional**. They exist so you can follow a complete example without being tied to a real repository. Always check a real project's own CONTRIBUTING guide, because every project has its own rules.

Contents

1 Why open source, and why now 2
2 How open source actually works 4
3 Ways to contribute (it is not just code) 7
4 Choose a project that fits you 9
5 From scratch: your first contribution 12
6 Reading an unfamiliar codebase 17
7 Communication, review and etiquette 19
8 Roadmap to mastery 22
9 Your first 14 days 27
10 Resources and glossary 29

CHAPTER 01

Why open source, and why now

Motivation, myths, and where you stand today

Open-source software is software whose source code is published under a licence that lets anyone **use, study, modify and share** it. You already depend on it every day: the Linux kernel behind most servers and Android phones, Git itself, Node.js, Python, React, Firefox, and the open-source core of VS Code are all built in public by communities of volunteers and companies.

FOR YOU **Where you stand today**

You already know Git and GitHub. This is the biggest tooling hurdle for most newcomers and you have cleared it.

At ~30% of frontend you can read HTML and CSS, follow basic JavaScript, and use browser DevTools. That unlocks documentation, UI and accessibility fixes, and small bug fixes.

At ~30% of system design you can follow a README's architecture section: what runs in the browser, what runs on a server, how they talk. That is enough to understand *where* a change belongs.

Six reasons to start in your first year

Reason	What it means in practice
Learn from real code	Textbook programs are small and clean. Production code has history, edge cases, tests, and trade-offs. Reading it is the fastest way to leave the beginner plateau.
Free mentorship	Every pull request is reviewed by someone more experienced, for free. A single thoughtful review can teach more than a week of tutorials.
Proof of work	A merged pull request is public, verifiable evidence that you can read unfamiliar code, communicate and finish work. It speaks louder than a line on a CV.
Industry-grade habits	Issues, branches, code review, continuous integration, semantic commits: this is how most software teams operate. You practise it before your first job.
Network and opportunities	Maintainers write recommendations, and programmes such as Google Summer of Code, Outreachy, LFX Mentorship and the MLH Fellowship recruit from active contributors.
Ownership and impact	Your fix may be used by thousands of developers. That feeling is hard to get from coursework.

Myths versus reality

Myth	Reality
"I must be an expert first."	Many valuable contributions need no deep expertise: fixing documentation, reproducing bugs, improving accessibility, adding tests.
"My code has to be perfect."	Reviewers expect iteration. A first PR that needs three rounds of feedback is completely normal and is where the learning happens.
"Maintainers are unfriendly."	Most are busy volunteers. A clear, polite, well-scoped pull request is a gift to them. Hostility is rare and is covered by codes of conduct.
"Only famous projects matter."	Small and medium projects often need help more, respond faster, and give you more visible ownership.
"It will eat all my study time."	Three to five focused hours a week is enough to land your first pull request within a month.
"I'm only 30% through my course."	That is already enough for a first pull request. Documentation, HTML/CSS fixes and small JavaScript bugs are within reach today.

CHAPTER 02

How open source actually works

The cast, the paperwork and the fork-and-pull model

Before you contribute, it helps to know who is who and what each file in a repository is trying to tell you. Open-source projects look chaotic from outside but follow a small number of conventions.

The cast of characters

Roles overlap: every maintainer once started as a user, then a contributor.

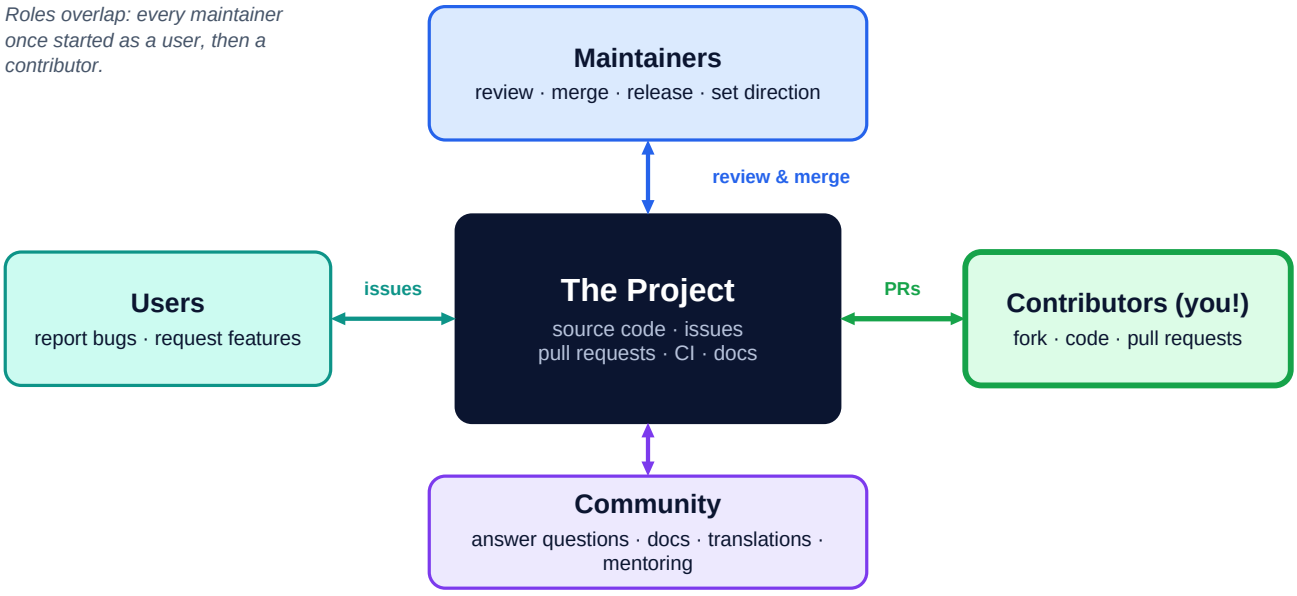


Figure 1. The open-source ecosystem. Information flows both ways; the three rule documents at the bottom apply to everyone.

Role	What they do	How you relate
User	Uses the software, reports bugs, asks for features.	You are already one, which gives you real insight into what is confusing or broken.
Contributor	Anyone who improves the project: code, docs, design, translation, triage, answers.	This is the role you are stepping into.
Maintainer	Reviews and merges contributions, triages issues, cuts releases, sets direction. Often a volunteer.	Your reviewer. Be kind to their time.
Owner / foundation	Holds the repository or trademark. Larger projects sit under foundations such as Apache, Linux Foundation or CNCF.	Sets governance and licence rules.

Files and features that tell you how a project works

File / feature	What it tells you	Why you care
`README.md`	What the project does, how to install and run it.	Your first stop. Run it locally before anything else.
`CONTRIBUTING.md`	How to set up, coding style, branch and commit rules, how to submit.	The most important file for you. Follow it exactly.
`CODE_OF_CONDUCT.md`	Behaviour expected in the community and how problems are handled.	Read once; it protects you too.
`LICENSE`	What others may do with the code (see licences below).	Defines what you are agreeing to when you contribute.
`.github/ templates`	Forms for bug reports and feature requests, and a checklist pre-filled in every new PR.	Complete them fully; maintainers notice.
`Issues / Discussions`	Bugs, tasks, ideas, questions. Labels group them.	Where you find and claim work.
`CI (GitHub Actions)`	Automatic tests and linters that run on every PR.	A red cross means fix it before asking for review.
`CODEOWNERS / SECURITY.md`	Who reviews which files; how to report vulnerabilities privately.	Know who to expect feedback from; never report security bugs publicly.

The fork-and-pull model

You almost never have permission to push directly to someone else's repository. Instead, open source uses **fork and pull**: you create your own server-side copy (a *fork*), work there, and ask the maintainers to *pull* your changes into the original (*upstream*) repository via a pull request. Git calls your fork **origin** and the official repo **upstream**. Because you already know Git, the only new idea here is having two remotes and keeping them in sync.

You never push to upstream. Every change travels through your fork and arrives as a pull request.

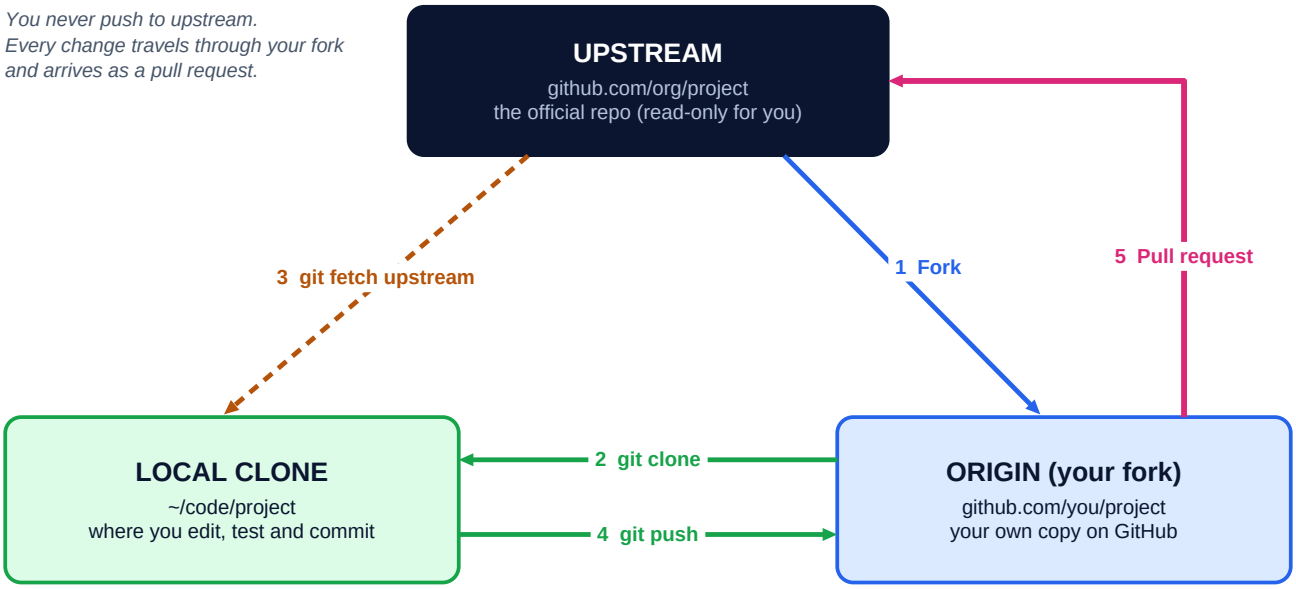


Figure 2. Remotes in the fork-and-pull model. The dashed orange arrow keeps your copy up to date with the official project.

Some projects use other workflows (the Linux kernel and Git itself review patches on mailing lists; teams with write access use shared branches). For GitHub-hosted projects as a newcomer, fork-and-pull is the norm.

Licences, CLAs and sign-offs

Licence	Type	In plain words
MIT / BSD	Permissive	Do almost anything, including closed-source use, as long as the licence notice stays.
Apache 2.0	Permissive + patent grant	Like MIT, with an explicit patent licence from contributors.
MPL 2.0	Weak copyleft	Changes to MPL-licensed *files* must be shared; other files can stay separate.
GPL 3.0	Strong copyleft	Derived works must also be GPL when distributed.
CC BY / CC BY-SA	Content licences	Common for documentation and images rather than code.

Some projects ask you to sign a **CLA** (Contributor License Agreement) through a bot, or to certify the **DCO** (Developer Certificate of Origin) by adding a sign-off line to each commit with `git commit -s`. Both simply state that you have the right to submit your work.

> **WATCH OUT** **Never copy code you do not have the right to copy**
> Do not paste code from Stack Overflow answers, other repositories or AI tools unless you understand it and its licence is compatible with the project's. When in doubt, write it yourself and cite the idea, not the code.

CHAPTER 03

Ways to contribute (it is not just code)

A menu of options and a ladder to climb

The most common misconception is that contributing means writing a clever feature. In practice, the majority of valuable work is quieter. Pick the types that match your current skills, then climb.

Type	Examples	Difficulty	Fit now?
Documentation	Fix typos and broken links, clarify setup steps, add usage examples, update outdated screenshots.	●○○○○	Yes, ideal first PR
Triage	Reproduce reported bugs, add missing details, find duplicates, suggest labels.	●●○○○	Yes
Accessibility and UX	Add alt text and ARIA labels, check colour contrast, fix keyboard navigation and focus rings.	●●○○○	Yes, matches frontend skills
Translation	Translate docs or interface strings into your language; review others' translations.	●○○○○	Yes
Tests	Add unit tests for untested behaviour, turn a bug report into a regression test.	●●●○○	Soon, after a few PRs
Bug fixes	Fix layout bugs, small logic errors, wrong error messages.	●●●○○	Yes for CSS/HTML/small JS
Features	Add a new component, option or integration, usually after a design discussion.	●●●●○	Later (Phase 3)
Tooling and CI	Improve build scripts, linters, GitHub Actions, developer experience.	●●●●○	Later
Code review	Read others' PRs, test them locally, leave kind and specific feedback.	●●●○○	Start as an observer
Community	Answer questions in Discussions or chat, welcome newcomers, write blog posts.	●●○○○	Yes
Security	Report vulnerabilities privately following SECURITY.md.	●●●●○	Report, never publish

The contribution ladder

Think of open source as a staircase. Each step builds trust with maintainers and teaches new skills. The timings under each step are typical for a student contributing a few hours a week; they are guides, not deadlines.

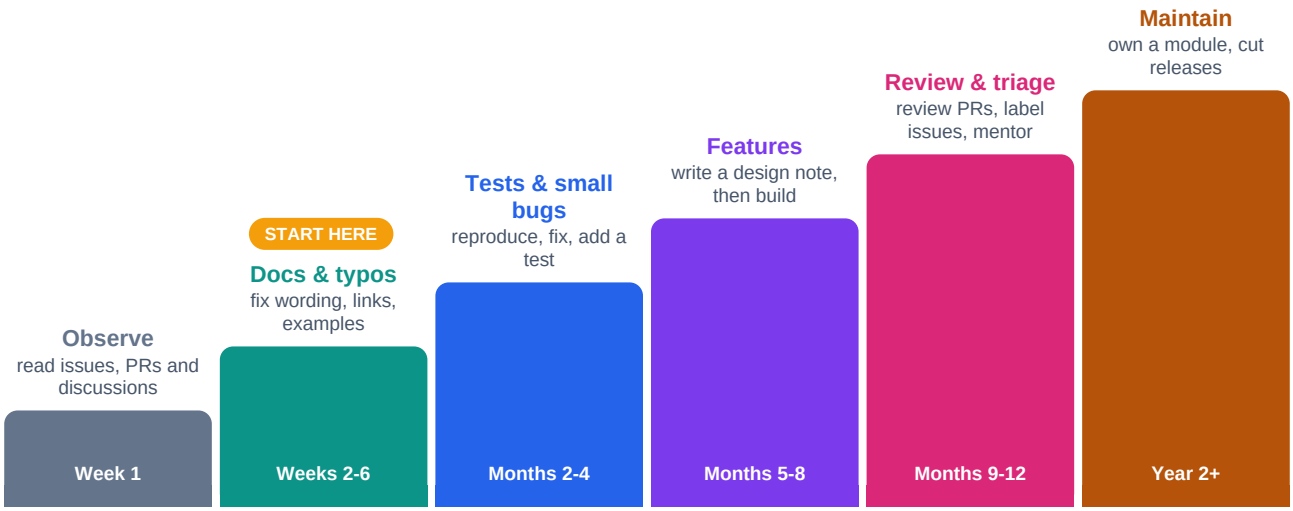


Figure 3. The contribution ladder from observer to maintainer.

Good first contributions at your current level

- Fix a typo, a dead link or an outdated command in the README or docs.
- Improve alt text, ARIA labels, heading order or colour contrast in a UI library or documentation site.
- Fix a CSS bug reported on a specific browser or screen size.
- Add a missing example, code comment or screenshot to a component's documentation.
- Reproduce an open bug, then comment with exact steps, browser and version. Triage is valuable.
- Translate a page of documentation into Hindi, Tamil, Bengali or any language you speak well.

What to avoid at first

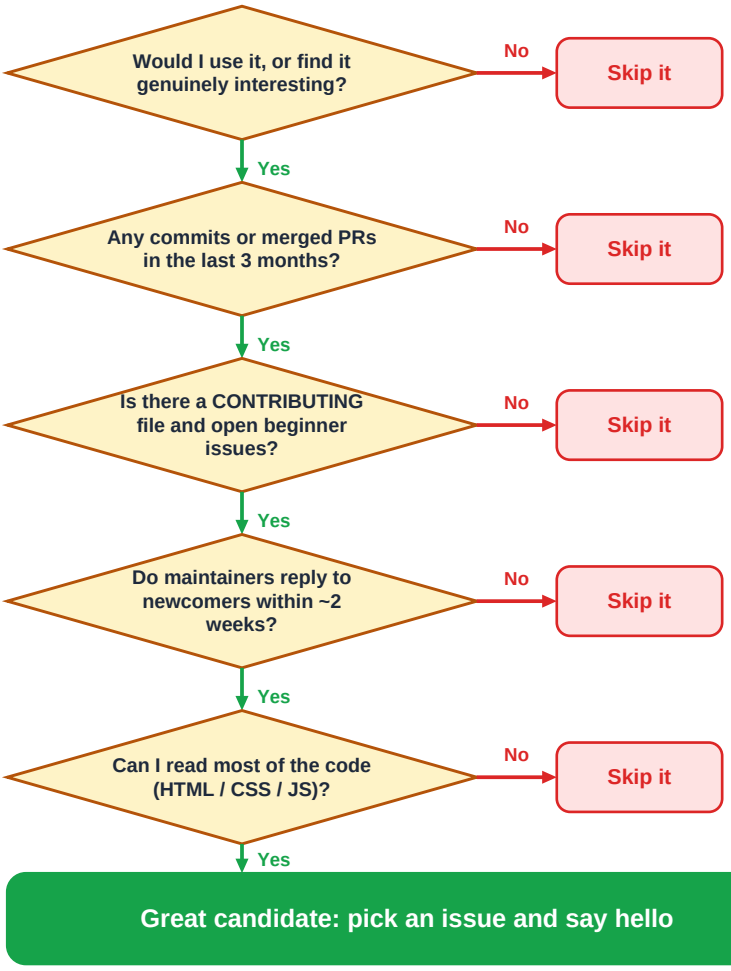
- **Large refactors or rewrites** that nobody asked for. They are hard to review and almost always rejected.
- **Unrequested features.** Open an issue and discuss first; otherwise you may build something the project does not want.
- **Drive-by mass PRs** (changing whitespace in fifty files, bulk renames) just to collect a count. Maintainers see this as spam.
- **Taking an issue and disappearing.** If life gets busy, say so and release the issue.

CHAPTER 04

Choose a project that fits you

A decision flow, a health check and places to look

The project you choose shapes your whole experience. A responsive, welcoming project with clear issues can turn a nervous beginner into a regular contributor; an abandoned one wastes weeks. Use the flowchart below on every candidate before investing time.



HOW TO CHECK
Motivation carries you through the dull parts. Start with tools already in your own package.json or sites you visit daily.

HOW TO CHECK
Open the Commits page and Insights > Pulse. A silent repository means nobody will review your pull request.

HOW TO CHECK
Look for the labels good first issue, help wanted, documentation, or a Contributing section in the README.

HOW TO CHECK
Open five recent issues and PRs. Are there human replies? Is the tone welcoming? How long did first PRs wait?

HOW TO CHECK
Browse src/. If the stack is unfamiliar, start with a docs or UI issue and grow into the rest.

Skipped a project? Bookmark it and revisit when you know more.

Figure 4. Project-selection flowchart with a quick way to check each question.

Read a repository like a doctor reads a chart

Open any candidate and scan the six signals marked in the figure. Together they take about five minutes and save weeks of frustration.

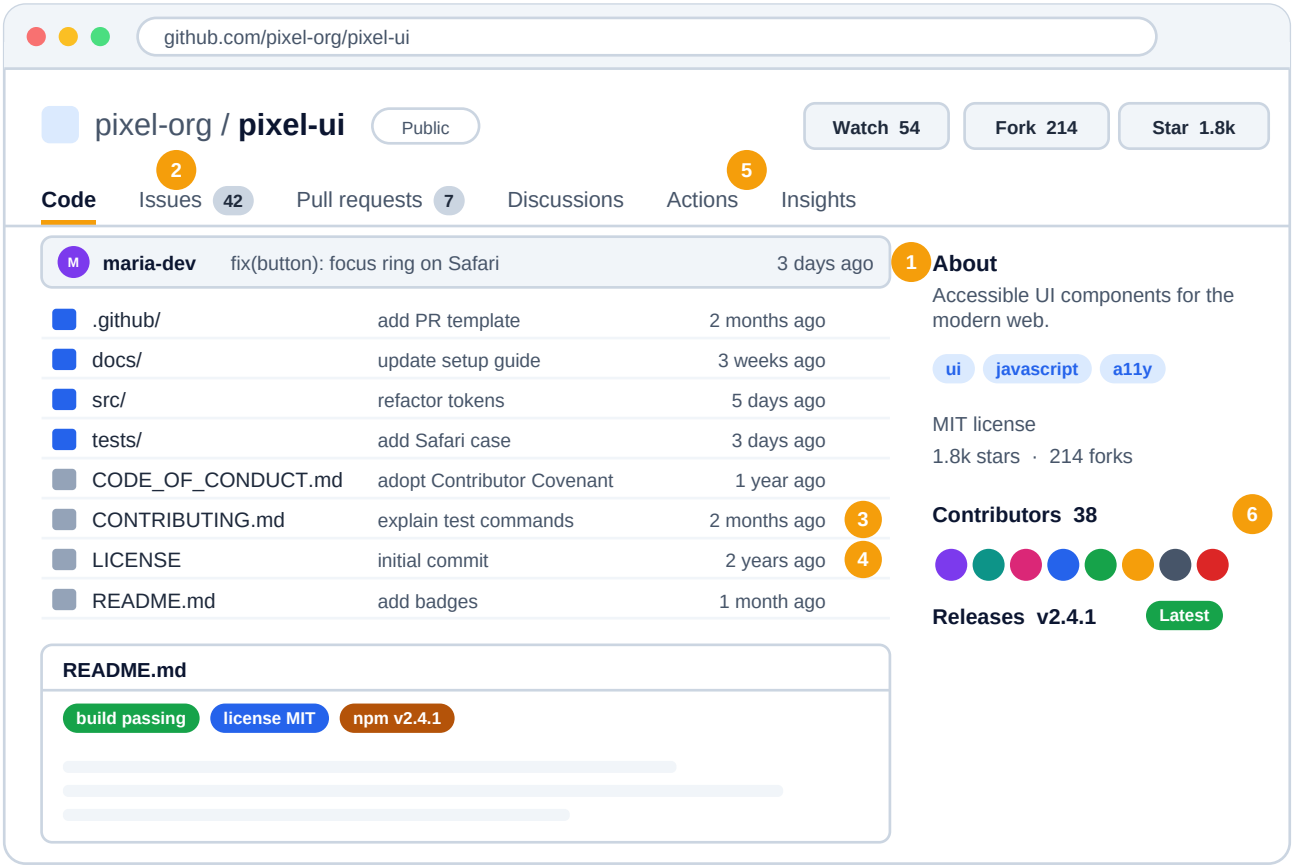


Figure 5. Anatomy of a repository home page (fictional project). The numbered markers are explained in the table below.

#	Signal	Green flag	Red flag
1	**Recent activity**	Commits within days or weeks; releases in the last year.	Last commit over six months ago.
2	**Issues and PRs**	Open issues have comments; PRs get reviewed and merged.	Hundreds of unanswered issues; PRs open for months.
3	**CONTRIBUTING**	Clear setup, style and PR instructions.	Missing, or contradicts the code.
4	**Licence**	A standard OSI-approved licence.	No licence file (legally, you have no permission to reuse the code).
5	**Automation**	CI runs tests and lint on every PR.	No tests; reviewers must check everything by hand.
6	**Community**	Many contributors, not just one or two.	A single maintainer who is also the only reviewer.

Where to find projects and issues

GitHub's search bar understands filters. These two queries are good starting points (paste them into the issues search and change the language or label):

GITHUB ISSUE SEARCH

```
is:issue is:open label:"good first issue" language:javascript no:assignee
is:issue is:open label:"help wanted" label:documentation no:assignee
```

Resource	What it offers
Your own package.json	Every dependency is an open-source project you already use. Run `npm ls --depth=0` and browse their repositories.
GitHub Topics	Browse the topics *good-first-issue*, *hacktoberfest* or *frontend* and sort by recently updated.
firstcontributions/first-contributions	A practice repository where you make a harmless PR to learn the flow safely.
up-for-grabs.net, goodfirstissue.dev, CodeTriage	Curated directories of beginner-friendly issues across projects; CodeTriage can email you open issues from repos you choose.
awesome-for-beginners	A community list of projects that welcome newcomers, grouped by language.
Documentation projects	MDN Web Docs, freeCodeCamp and the docs sites of React, Vite or Docusaurus are open and frontend-friendly. Check each project's current CONTRIBUTING guide first.

Programmes that organise open-source learning

Programme	What it is	Good to know
Hacktoberfest	A month-long October event encouraging pull requests to participating repositories.	Quality matters; spammy PRs are rejected. Check the current rules.
Google Summer of Code	Mentored, stipend-based projects with open-source organisations over a summer.	Start contributing months earlier; mentors prefer known faces.
Outreachy	Paid remote internships with open-source communities for people underrepresented in tech.	Requires contributions during an application period.
LFX Mentorship, MLH Fellowship	Mentored projects under the Linux Foundation, and a fellowship with an open-source track.	Eligibility and dates change each year.
Student programmes in India	Community-run seasons such as GirlScript Summer of Code or Social Winter of Code.	Verify the current edition on the official site.

GOOD TO KNOW **Always verify**

Programme rules, dates and stipends change every year. Treat the table above as a map of what exists, and check each programme's official page for current details.

CHAPTER 05

From scratch: your first contribution

Twelve steps, with real commands, from nothing to a merged PR

This is the heart of the guide. We follow one complete contribution on the fictional project **pixel-ui**: a keyboard-accessibility bug (#123: the button loses its focus ring in Safari). Replace the names with the real project you chose in Chapter 4 and the steps stay the same.

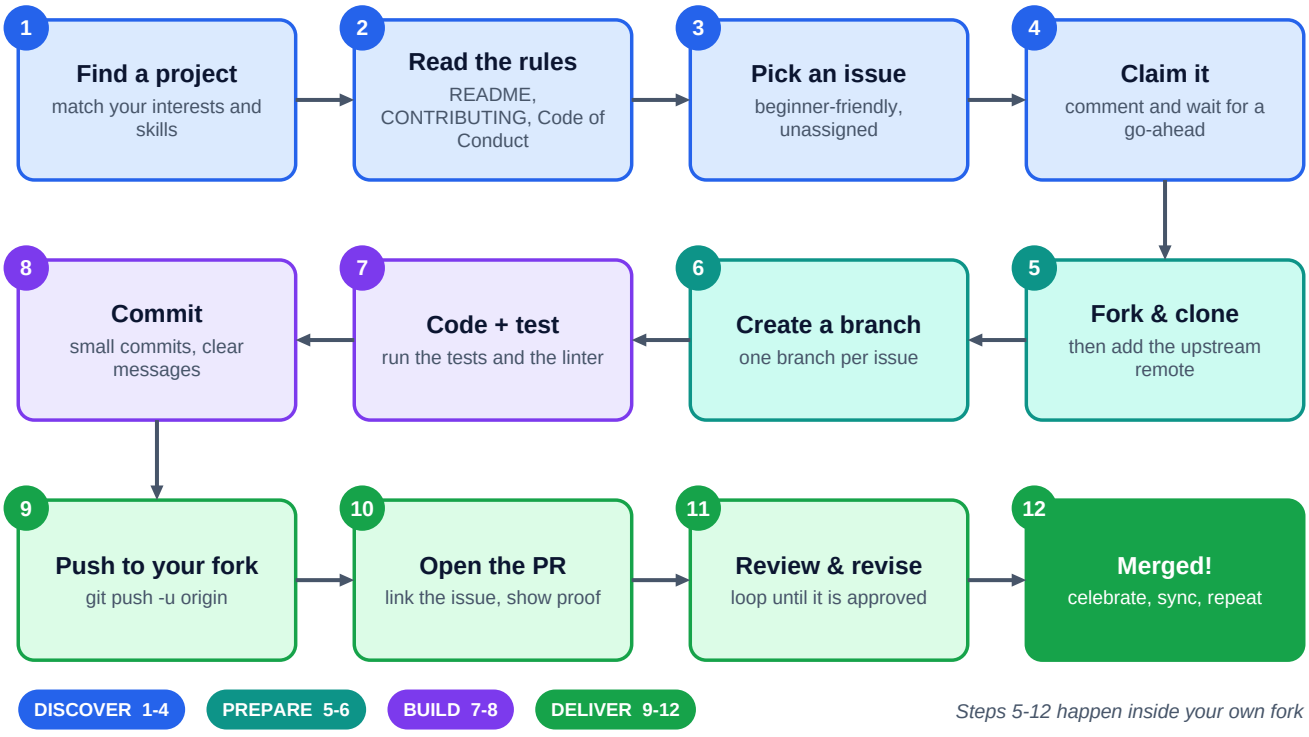


Figure 6. The twelve-step contribution lifecycle. Steps 1 to 4 are about people and rules; steps 5 to 12 are about Git and code.

Step 0 · One-time setup (30 minutes)

- **GitHub profile:** add a photo, a short bio and enable two-factor authentication. Maintainers do look at who is contributing.
- **Git identity:** use the private GitHub no-reply address if you do not want your personal email in every commit.
- **Tooling:** install the current Node.js LTS, a code editor with ESLint and Prettier extensions, and optionally the GitHub CLI (gh).

ONE-TIME SETUP

```
$ git config --global user.name "Your Name"
$ git config --global user.email "12345+yourname@users.noreply.github.com"
$ gh auth login            # optional: lets you fork and open PRs from the terminal
```

Steps 1 to 4 · Discover, understand and claim

1. Find a project using Chapter 4's flowchart. **2. Read the rules** before touching code: README, CONTRIBUTING, Code of Conduct, issue and PR templates. Then **run the project locally** exactly as the README says. If you cannot get it running, ask for help in an issue or the chat; fixing the setup docs is itself a great first contribution.

3. Pick an issue. Look for labels such as *good first issue* or *help wanted* that are open, unassigned, recent and clearly described. Skim the comments to ensure nobody else is already working on it. **4. Claim it** with a short comment and wait for the maintainer's go-ahead before writing code.



Figure 7. A well-chosen issue and a good claim. (1) beginner-friendly labels; (2) clear reproduction steps; (3) your comment shows you reproduced the bug; (4) the maintainer's go-ahead.

> **TIP** **What makes a good claim comment**
> Say what you understood, show you tried to reproduce it, outline your plan in one sentence, and ask if the approach is acceptable. Avoid a bare "Can I work on this?" which tells the maintainer nothing and is often ignored.

Steps 5 and 6 · Fork, clone and branch

Forking creates your copy on GitHub; cloning brings it to your machine; adding an *upstream* remote lets you receive the project's latest changes (see Figure 2). Always work on a **new branch** named after the issue, never on your fork's `main`.

FORK, CLONE, BRANCH

```
# Option A: GitHub CLI forks, clones and wires up the upstream remote for you
$ gh repo fork pixel-org/pixel-ui --clone

# Option B: click "Fork" in the browser, then:
$ git clone https://github.com/<you>/pixel-ui.git
$ cd pixel-ui
$ git remote add upstream https://github.com/pixel-org/pixel-ui.git
$ git remote -v            # origin = your fork, upstream = official repo

$ npm install              # follow CONTRIBUTING.md for the exact commands
$ npm test                 # make sure everything passes BEFORE you change anything
$ git switch -c fix/button-focus-ring-123
```

Steps 7 and 8 · Code, test and commit

Make the **smallest change that solves the issue**. Run the tests and linter locally, view your change in the browser, and add or update a test that would have caught the bug. For our example the fix is only a few lines of CSS:

THE CHANGE

```
/* src/button.css */
.btn:focus-visible {
  outline: 2px solid var(--pixel-focus, #2563eb);
  outline-offset: 2px;
}
```

Commit with a message that follows the project's convention. Many projects use **Conventional Commits**: a type (`fix`, `feat`, `docs`, `test`), an optional scope in brackets, then a short imperative summary. If the project requires a DCO sign-off, add `-s`.

COMMIT

```
$ git add src/button.css tests/button.test.js
$ git commit -s -m "fix(button): show focus ring on keyboard focus in Safari" -m "Closes #123"
$ npm test && npm run lint     # run again, then proceed
```

Keeping your branch current

While you work, other people's changes are merged upstream. Before opening the PR (and again if a reviewer asks), bring your branch up to date. Follow the project's preference: some want a *rebase*, others prefer merging `main` into your branch.

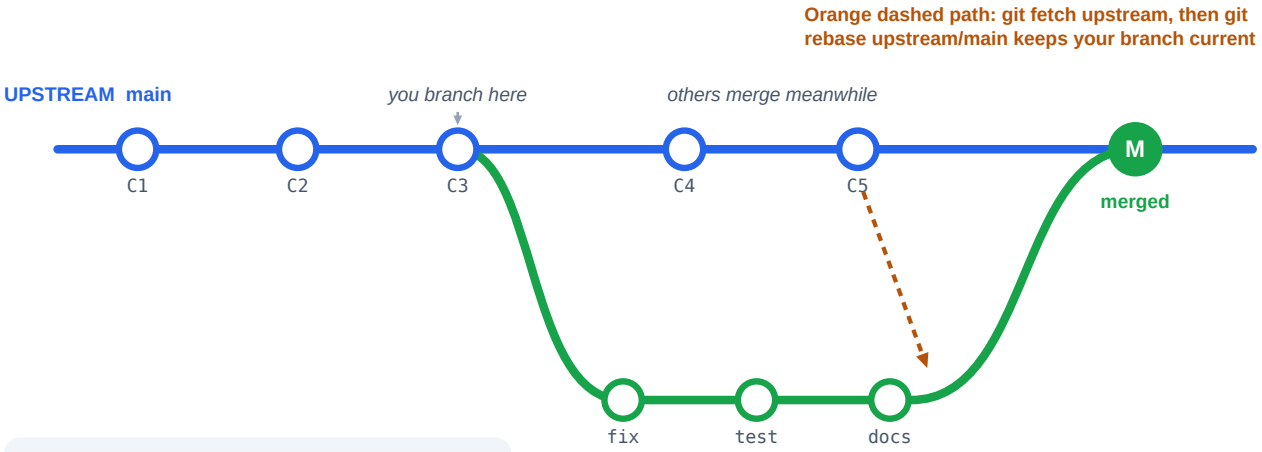


Illustration: each circle is a commit. Your branch lives on your fork and joins upstream only through a reviewed pull request.

Figure 8. Your branch diverges from upstream main, others keep merging, and your PR joins them at the end.

SYNC

```
$ git fetch upstream
# sync your fork first
$ git switch main && git merge --ff-only upstream/main && git push origin main
$ git switch fix/button-focus-ring-123
$ git rebase upstream/main        # replay your commits on top of the latest main
$ git push --force-with-lease     # only on your own PR branch, never on shared branches
```

Steps 9 and 10 · Push and open the pull request

PUSH AND OPEN

```
$ git push -u origin fix/button-focus-ring-123
$ gh pr create --fill            # or click "Compare & pull request" on GitHub
```

Your PR description is a user interface for the reviewer. Fill in the template completely. Link the issue with **Closes #123** so it closes automatically on merge, explain *what* and *why*, tell the reviewer *how to test*, and attach before/after screenshots for any visual change.

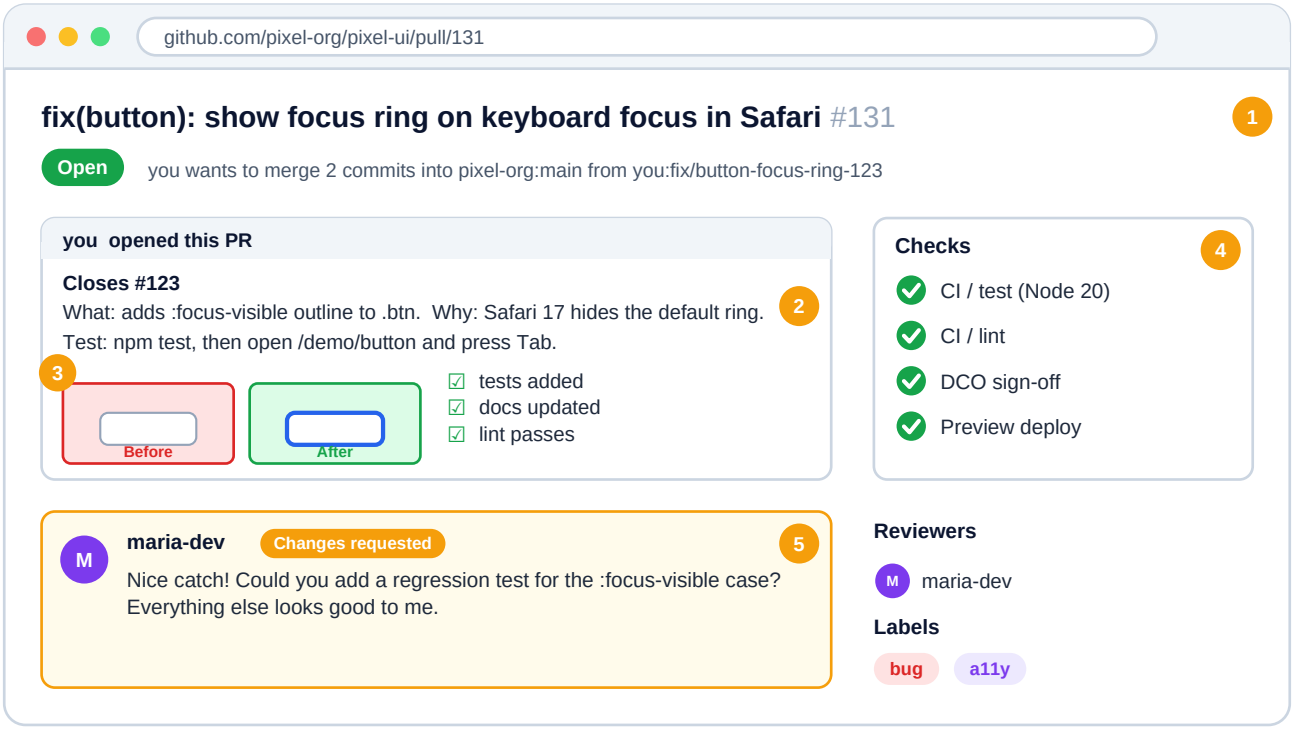


Figure 9. A pull request. (1) conventional title; (2) issue link and explanation; (3) visual proof; (4) automated checks; (5) reviewer feedback.

A STRONG PR DESCRIPTION

```
## What
Adds a visible :focus-visible outline to .btn.

## Why
Safari 17 does not draw a default focus ring, so keyboard users
cannot see which button is focused. Closes #123.

## How to test
1. npm install && npm test
2. npm run dev, open /demo/button in Safari and press Tab.

## Screenshots
Before / After attached.

## Checklist
- [x] Tests added   - [x] Docs updated   - [x] Lint passes
```

Steps 11 and 12 · Review, revise, merge

Within minutes, automated checks run; within days or weeks a human reviews. Expect questions and requested changes. Treat each comment as a gift, reply to every thread, and push fixes to the *same branch*; the PR updates automatically. Chapter 7 covers the etiquette in detail.

After the merge: sync your fork, delete the branch, and take a moment to note what you learned. You are now a contributor.

CLEAN UP AFTER MERGE

```
$ git switch main && git fetch upstream
$ git merge --ff-only upstream/main && git push origin main
$ git branch -d fix/button-focus-ring-123
$ git push origin --delete fix/button-focus-ring-123
```

Command cheat sheet

Goal	Command
See your remotes	`git remote -v`
Start a branch	`git switch -c fix/short-description-123`
Get latest upstream changes	`git fetch upstream`
Update fork's main	`git merge --ff-only upstream/main` then `git push origin main`
Re-base your branch	`git rebase upstream/main`
Amend the last commit	`git commit --amend` (then `git push --force-with-lease`)
Check out someone's PR to test	`gh pr checkout 131`
Undo local edits to a file	`git restore path/to/file`

CHAPTER 06

Reading an unfamiliar codebase

A method for not drowning, and where system design shows up

The scariest moment is opening a repository with hundreds of files. The trick is to stop trying to understand everything. Your goal is to understand **just enough** to make one change safely.



Figure 10. A six-step method for reading code. Repeat only as deeply as your issue requires.

Tools that make it faster

Tool	Use it to
Editor global search (or `rg` / `grep -rn`)	Search for an error message, class name or text from the UI to land exactly where it is produced.
Browser DevTools	Inspect the element, see which CSS rule applies, set breakpoints in JavaScript, watch network requests.
`git log -p -- file` and `git blame file`	See who changed a line and the pull request that explains why.
Closed issues and PRs	Find how similar bugs were fixed before; copy the style and the testing approach.
A debugger or `console.log`	Confirm what you think the code does by watching it run.

Your system-design lessons, in the wild

The system-design material you are studying is not abstract: every concept appears in real repositories. Looking for it is the best way to make the theory stick, and it builds the intuition you need to move from small fixes toward features.

Concept you are learning	Where you will see it in a frontend project
Client-server model	API client modules, `fetch` calls, mock servers used in tests.
Layers and separation of concerns	Folders such as components, hooks, services, utils; UI code kept apart from data code.
State and data flow	Stores, context providers, props and events that move data between components.
Caching	HTTP cache headers, memoisation, service workers, build and CI caches.
Build pipeline	Bundler config (Vite, webpack, Rollup) turning `src/` into `dist/`.
Dependency management	`package.json`, lockfiles, semantic-version ranges, monorepo workspaces.
CI/CD	`.github/workflows`: test, build, then publish on release.

Concept you are learning	Where you will see it in a frontend project
API design	A UI library's public props, events and options are its API; changing them needs deprecation and a version bump.
Performance and scale	Bundle-size budgets, lazy loading, Lighthouse checks in CI.

TRY IT **Architecture sketch exercise (60 minutes)**

Pick a project you chose in Chapter 4. Draw one page showing its main parts and how data moves between them: for a UI library that might be *source components, build, tests, docs site, npm package*. Under the diagram, write five questions you still have.

Post your sketch (or its best question) in the project's Discussions or chat. Many maintainers love this and some will add it to the contributor docs.

CHAPTER 07

Communication, review and etiquette

How to be someone maintainers are happy to work with

Technical skill gets your first pull request written. Communication gets it merged, and gets your second one welcomed. Remember that maintainers are often volunteers reviewing code in their free time.

Principles

- **Search before you ask.** Check the docs, open and closed issues, and Discussions. Mention what you already tried.
- **Keep PRs small and focused.** One concern per PR, ideally under a couple of hundred changed lines. Small PRs are reviewed faster.
- **Be specific.** Include versions, steps, expected and actual results, screenshots or error text.
- **Respond promptly.** Reply to review comments within a few days, or say when you will be back.
- **Be patient.** Wait one to two weeks before a single, polite reminder. Never ping repeatedly or in unrelated threads.
- **Accept "no" gracefully.** A closed PR is feedback about fit, not about you. Ask what would make a future one acceptable.

Message templates you can adapt

Situation	Template
Claiming an issue	*Hi! I'd like to work on this. I reproduced it on Safari 17 and I think the cause is the :focus-visible rule in button.css. My plan: add an explicit outline and a regression test. Is that approach okay?*
Asking for help	*I'm trying to run the docs site locally. On Node 20 / Windows 11,* `npm run dev` *fails with the error below. I already checked CONTRIBUTING.md and issue #98. Could you point me in the right direction?*
Responding to review	*Thanks for the review! I pushed a fix in the latest commit. I kept the name* `focusColor` *because it matches the other tokens, but I'm happy to change it if you prefer.*
Polite reminder	*Hi @maintainer, just checking whether there's anything I should change to help move this PR forward. Happy to adjust. Thank you for your time!*
Releasing an issue	*I'm sorry, I no longer have time to work on this. Please unassign me so someone else can pick it up. Here's what I found so far...*

The review loop

Every PR passes through two filters: machines first, then humans. Understanding the loop removes most of the anxiety.

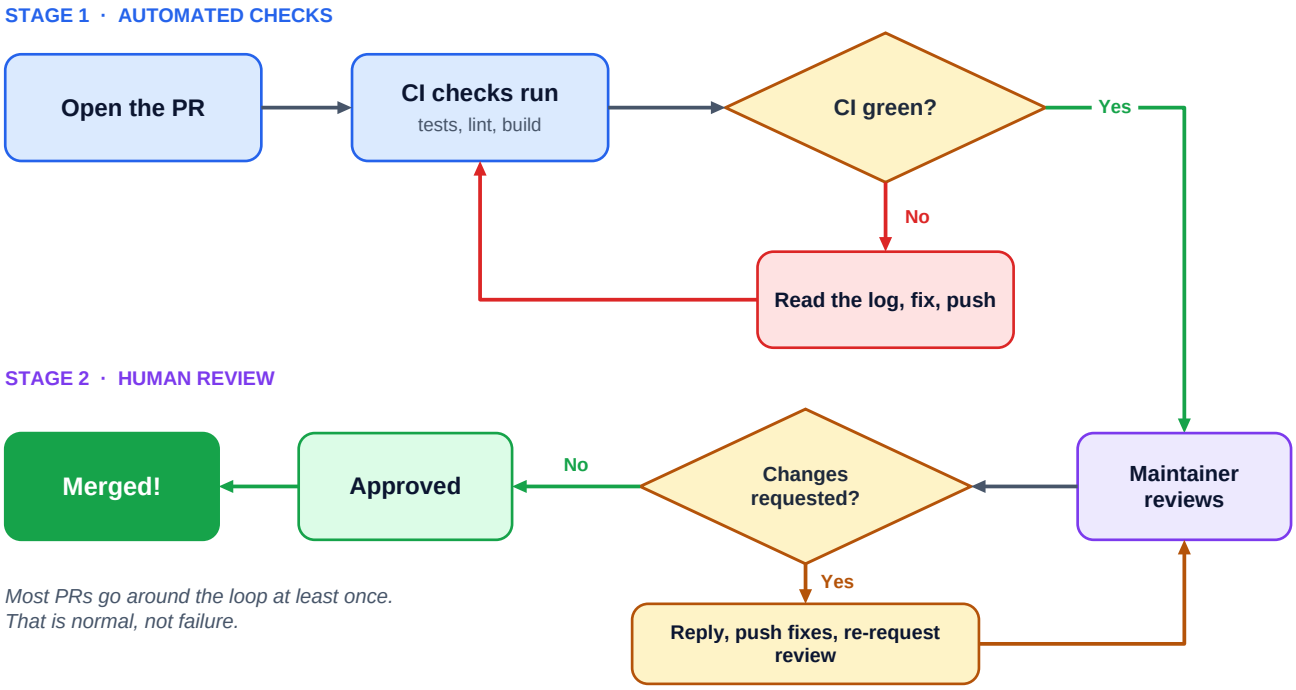


Figure 11. The review loop. Going round it several times is the normal path to a merge.

Receiving feedback well

- Read every comment fully before replying. Assume good intent.
- If you agree, fix it and reply "Done" with the commit. If you disagree, explain your reasoning calmly and be ready to be persuaded.
- If you do not understand a comment, ask; reviewers would rather clarify than see a wrong guess.
- Mark conversations as resolved only if the project's convention lets the author do so; otherwise leave it to the reviewer.

When you are stuck

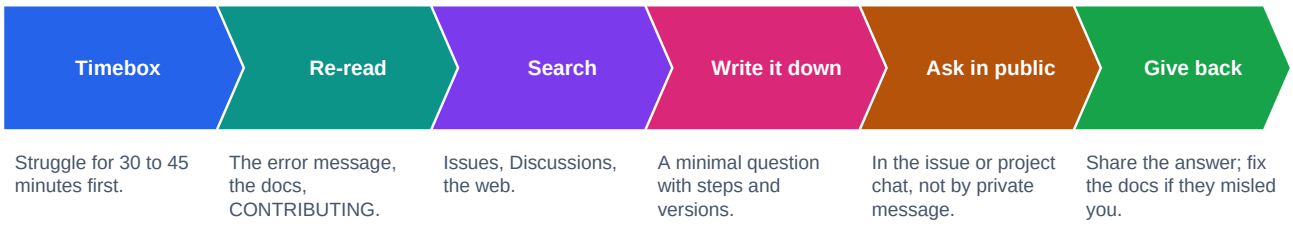


Figure 12. A six-step approach to asking for help effectively.

Using AI assistants responsibly

AI coding tools can help you understand unfamiliar code or draft tests, but a pull request carries **your** name. Policies vary: some projects forbid AI-generated contributions, some ask you to disclose their use, and many maintainers are overwhelmed by low-effort, unverified submissions.

- Read the project's policy before using any assistant for contributions; follow disclosure rules.
- Never submit code you cannot explain line by line. Reviewers will ask.
- Run the tests and try the change yourself. Do not trust generated output blindly.

- Write the PR description and issue comments in your own words; boilerplate text wastes reviewers' time.
- Use assistants as a tutor ("explain this function") more than as an author.

Common mistakes and quick fixes

Mistake	Why it hurts	Fix
Working before claiming	Someone else may be doing the same, or the maintainer may not want the change.	Comment first and wait for a go-ahead.
Committing to `main` of your fork	Makes syncing and multiple PRs messy.	One branch per issue.
Huge PRs	Hard to review, so they sit for weeks.	Split into small PRs; one concern each.
Ignoring CI failures	Reviewers will not look at red builds.	Read the log, reproduce locally, push a fix.
Reformatting unrelated code	Noisy diff hides the real change.	Run the project's formatter only on files you changed.
Committing secrets or build files	Leaked keys, bloated repository.	Check `.gitignore` and run `git status` before committing.
Vanishing after feedback	The PR goes stale and is closed.	Reply within days; say when you will be back.
Arguing in comments	Hurts trust and violates most Codes of Conduct.	Disagree respectfully, then accept the maintainers' decision.

WATCH OUT **Security hygiene**

Never commit passwords, API keys or `.env` files. If you find a security vulnerability, do **not** open a public issue: follow the project's SECURITY.md or use GitHub's private vulnerability reporting.

CHAPTER 08

Roadmap to mastery

From first PR to maintainer, with learning tracks and checkpoints

Mastery in open source is the combination of three things: **technical depth** (you can solve hard problems in a codebase), **collaboration** (you review, communicate and mentor well) and **stewardship** (you care for a project's long-term health). The roadmap below grows all three at once and ties your coursework to real contributions.

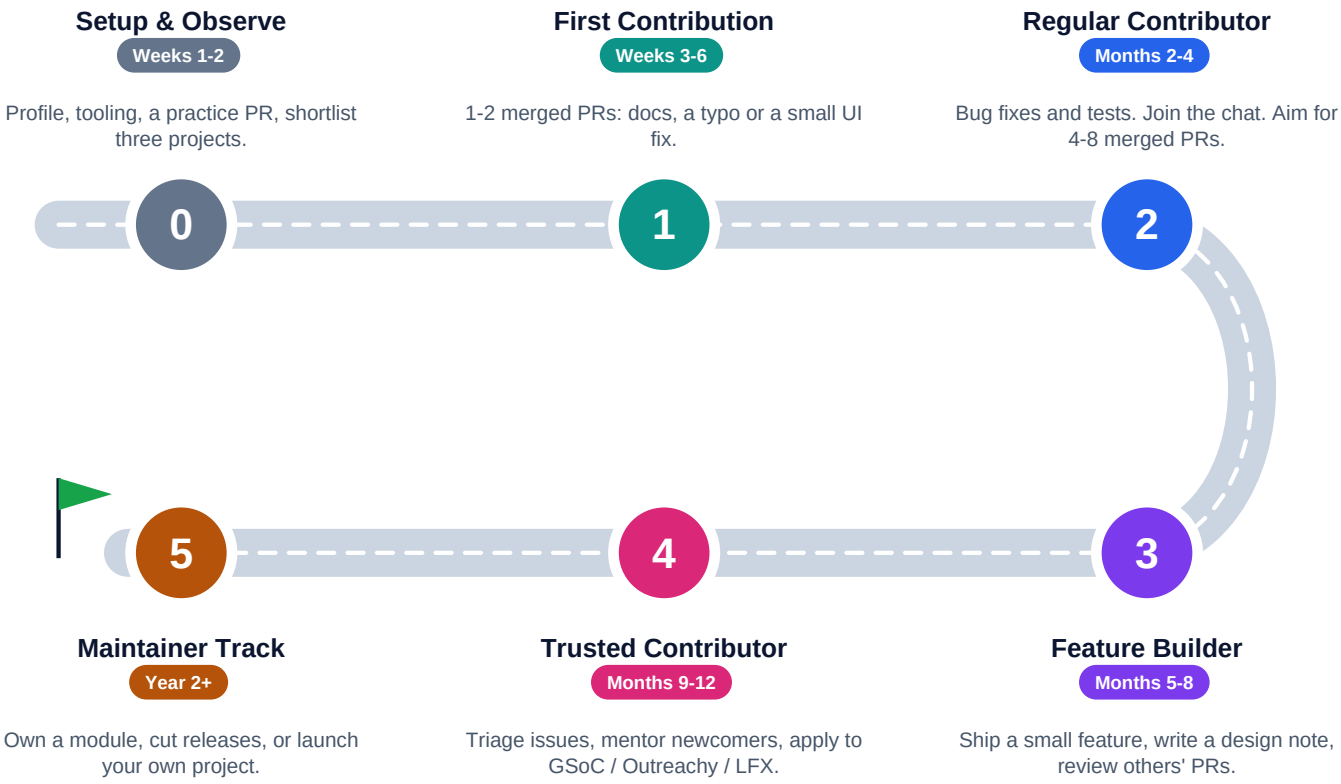


Figure 13. The six-phase roadmap. Timelines assume 3 to 5 hours per week; move faster or slower without guilt.

GOOD TO KNOW **How to read the roadmap**

Each phase has a **goal**, concrete **actions**, **learning** targets that match your frontend and system-design courses, and a **checkpoint**. Do not advance until you pass the checkpoint, and do not skip phases: trust is built step by step.

Phase 0 · Setup and Observe Weeks 1-2	
GOAL	Be ready to contribute and understand the landscape.
DO	• Polish your GitHub profile and enable two-factor authentication. • Install Node.js, a linter-aware editor and configure Git identity. • Make a practice PR in a beginner repository such as first-contributions. • Shortlist three projects with Figure 4; read their last ten merged PRs.
LEARN	**Frontend:** semantic HTML, browser DevTools, how CSS is applied. **System design:** what happens when a browser requests a page (HTTP request and response).
CHECKPOINT	Three candidate repositories run locally and you can describe each in two sentences.

Phase 1 · First Contribution Weeks 3-6	
GOAL	Land one merged pull request.
DO	• Choose a documentation, typo or small UI issue and claim it. • Follow Chapter 5 end to end; respond to review within 48 hours. • Write a short note on what you learned and what surprised you.
LEARN	**Frontend:** responsive CSS (flexbox, grid, media queries), accessibility basics (alt text, labels, contrast). **System design:** read the architecture section of the README and name the main components.
CHECKPOINT	One or two merged PRs and one complete review cycle experienced.

Phase 2 · Regular Contributor Months 2-4	
GOAL	Become dependable at small fixes and tests.
DO	• Fix reproducible bugs and add regression tests. • Join the project chat; answer one newcomer's question per week. • Keep a contribution journal (see Chapter 9).
LEARN	**Frontend:** ES6+ modules, async/await and fetch, npm scripts and tooling, unit testing (Jest or Vitest), cross-browser quirks. **System design:** REST and HTTP semantics, status codes, JSON APIs, caching basics.
CHECKPOINT	Four to eight merged PRs; maintainers recognise your username.

Phase 3 · Feature Builder Months 5-8	
GOAL	Own a small feature from idea to release.
DO	• Propose it in an issue; write a one-page design note (problem, options, trade-offs). • Implement with tests and docs; iterate on review. • Start reviewing other people's PRs: test locally, leave kind, specific comments.
LEARN	**Frontend:** a framework such as React or Vue, TypeScript, component design, state management, end-to-end tests (Playwright or Cypress). **System design:** modular design, API design, writing decision records, CI/CD pipelines.
CHECKPOINT	A feature shipped in a release; ten or more helpful review comments given.

Phase 4 · Trusted Contributor Months 9-12	
GOAL	Be someone maintainers rely on.
DO	• Triage issues weekly: reproduce, label, ask for missing details. • Mentor newcomers; improve CI, tests or contributor docs. • Apply to programmes (GSoC, Outreachy, LFX, MLH) and write a blog post or give a campus talk.
LEARN	**Frontend:** performance (Core Web Vitals), bundlers, design systems, internationalisation. **System design:** scalability patterns, observability basics, trade-off analysis.
CHECKPOINT	Triage or review rights granted, or accepted into a mentoring programme (a goal, not a guarantee).

Phase 5 · Maintainer Track Year 2+	
GOAL	Take responsibility for a project's long-term health.
DO	• Own a module or area; review PRs on a regular schedule. • Cut releases (semantic versioning, changelog) and handle security reports. • Or start your own project with good docs and a welcoming contributor guide.
LEARN	**Frontend:** library architecture, long-term API design, deprecation strategy. **System design:** distributed systems, project governance, sustainability.
CHECKPOINT	Maintainer or triage role, or a project of your own with outside contributors.

Parallel learning tracks: what your percentage unlocks

The percentages below are rough self-assessment bands, not exam scores. They show how your coursework progress translates into what you can do in open source.

Band	Frontend skills	System-design skills	Unlocks in open source
30-45% (you are here)	Semantic HTML, flexbox/grid, DOM events, basic fetch.	HTTP, client-server, REST, JSON.	Docs, CSS fixes, accessibility, triage.
45-60%	ES6+, modules, async code, npm, unit tests, DevTools profiling.	Databases, caching, authentication basics.	JavaScript bug fixes, tests.
60-75%	Framework + TypeScript, state, forms, end-to-end tests.	API design, queues, CDNs, CI/CD.	Features, design notes, reviews.
75-90%	Performance, accessibility audits, SSR and build tools, design systems.	Scalability, observability, writing RFCs, trade-offs.	Triage lead, tooling and CI work.
90%+	Framework and library internals.	Distributed systems, governance.	Maintain, lead, mentor.

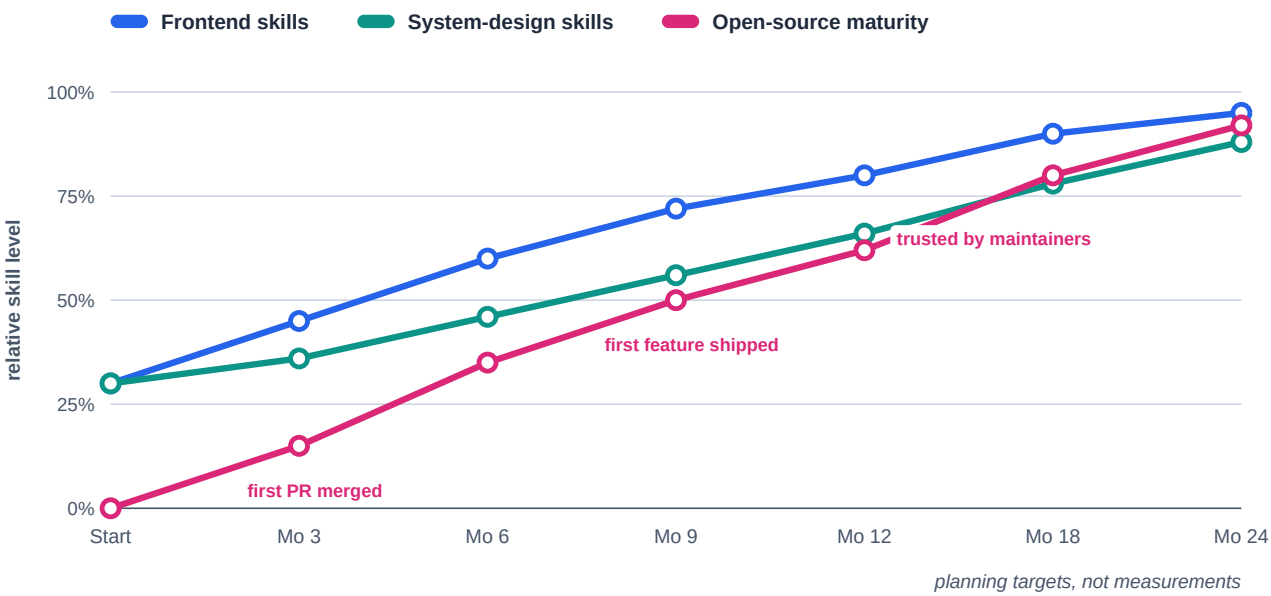


Figure 14. A planning target for how skills and contribution maturity can grow together over two years.

The mastery rubric

Level	Signals that you have reached it	Typical time
Newcomer	First PR merged; you follow CONTRIBUTING and ask well-formed questions.	Weeks 3-6
Contributor	Several merged PRs; you include tests; you respond quickly to review.	Months 2-4
Regular	Maintainers recognise you; you fix bugs with little guidance; you review others.	Months 5-8
Trusted / core	Triage or review rights; you propose designs and mentor newcomers.	Year 1-2
Maintainer	You merge, release, handle security and governance, and say "no" kindly.	Year 2+

A sustainable weekly rhythm (about 4 hours)

Time	Activity	Why
1 hour	Read code, issues and merged PRs of your project.	Builds the mental map and project taste.
2 hours	Work on your current issue: code, test, document.	Deliberate practice with feedback.
45 minutes	Review a PR, answer a question or triage an issue.	Teaches you how others think; builds trust.
15 minutes	Update your journal: what you did, learned, and what is next.	Makes progress visible and portfolio-ready.

WATCH OUT **Measure what matters**

Do not chase green squares or PR counts. Better signals are: reviews given, issues triaged, newcomers helped, documentation improved, and how long maintainers take to trust you with something bigger. Ten thoughtful contributions beat a hundred trivial ones.

Beyond contributing: starting your own project

When you can maintain well, you can launch something of your own. A welcoming project has these in place before the first announcement:

- A clear **README** (what it does, screenshot, quick start) and a standard **LICENSE** (see choosealicense.com).
- **CONTRIBUTING.md** and **CODE_OF_CONDUCT.md**, plus issue and PR templates.
- Labels such as *good first issue* with issues that are truly beginner-friendly and well described.
- **Automated tests and CI**, so contributors get fast, objective feedback.
- Tagged **releases** with a changelog and semantic versioning.
- A habit of thanking contributors publicly and responding within a few days.

Turning contributions into career capital

- **Profile:** pin your best repositories, add a profile README and link to merged PRs.
- **Write:** a short post per milestone ("How I fixed my first accessibility bug") compounds into visibility.
- **Resume bullets:** state the problem, your action and the result. Example pattern: "Fixed keyboard-focus bug in [project] (merged PR #N), improving usability for keyboard users in Safari."
- **References:** after sustained work, politely ask a maintainer whether they would be willing to vouch for you.

CHAPTER 09

Your first 14 days

A concrete plan, plus the checklists to use every time

Plans beat intentions. Here is a fortnight that takes you from nothing to an open pull request, assuming about one hour on weekdays and a longer block at the weekend.

Day	Action	Output
1	Set up profile, two-factor authentication, Git identity, Node and editor (Chapter 5, Step 0).	Ready environment
2	Make a practice pull request in a beginner repository such as first-contributions.	Muscle memory for fork, branch, PR
3-4	Use the Chapter 4 flowchart on 5 candidate projects. Keep the best 3.	Shortlist of 3
5	Read README, CONTRIBUTING and Code of Conduct of your top choice; run it locally.	Project runs on your machine
6	Read ten recently merged PRs and ten closed issues. Note the tone, labels and review style.	A feel for the culture
7	Rest, and write down which issue you would like to try and why.	One chosen issue
8	Post your claim comment (Chapter 7 template) and wait for the go-ahead.	Issue assigned to you
9-11	Fork, branch, code, test, commit. Keep the change small.	A working local change
12	Sync with upstream, push and open the pull request using the checklist below.	Open PR
13	Respond to automated checks and any early feedback.	Green CI, replies sent
14	Retrospective: what went well, what to improve. Choose your next issue.	Journal entry

Checklist before you click "Create pull request"

- ☐ I read CONTRIBUTING.md and followed its commit and branch conventions.
- ☐ The issue was assigned to me or the maintainer approved my plan.
- ☐ My branch is based on the latest upstream main and is named after the issue.
- ☐ The change is small, focused and does not include unrelated formatting.
- ☐ Tests pass locally; I added or updated tests where it makes sense.
- ☐ Linter and formatter pass; no debug code, secrets or build artefacts are included.
- ☐ I tested the change myself (in the browser, on the browsers or sizes that matter).
- ☐ The PR title follows the convention and the description fills in the template.
- ☐ I linked the issue with "Closes #N" and attached screenshots for visual changes.
- ☐ I reviewed my own diff on GitHub as if I were the reviewer.

Contribution journal template

Keep a simple file or note. It turns scattered effort into a portfolio, and it is the raw material for blog posts, resume bullets and programme applications.

JOURNAL ENTRY

```
Date:         2026-10-20
Project:      pixel-org/pixel-ui   (issue #123, PR #131)
What I did:   Added :focus-visible outline for .btn; wrote a regression test.
What I learned: Safari hides default focus rings; use :focus-visible instead of :focus.
Feedback received: Add a test; keep colour as a CSS variable.
Next step:    Look for another a11y issue; review one PR this week.
```

CHAPTER 10

Resources and glossary

Where to learn more, and the words you will meet

Recommended resources

Resource	Good for
opensource.guide (GitHub's Open Source Guides)	How to contribute, how to start a project, building communities and maintainer best practices.
GitHub Docs: Contributing to a project	Official walkthrough of forks, branches, pull requests and issues.
firstcontributions/first-contributions	A safe practice repository for your first pull request.
Pro Git (git-scm.com/book)	Deepening Git skills: rebase, history rewriting, advanced branching.
MDN Web Docs and **web.dev**	Authoritative frontend references, accessibility and performance guidance.
roadmap.sh (Frontend roadmap)	A visual map of frontend topics to pair with Phase learning targets.
System Design Primer (donnemartin/system-design-primer)	A free, structured introduction to system-design concepts.
choosealicense.com	Plain-language help comparing open-source licences.
conventionalcommits.org and **semver.org**	The commit-message and versioning conventions mentioned in this guide.

Glossary

Term	Meaning
Upstream	The original repository you forked from.
Origin	Your fork on GitHub; the default name of your main remote.
Fork	Your own server-side copy of someone else's repository.
Pull request (PR)	A proposal to merge your branch's changes into another branch or repository.
Issue	A tracked bug, task, question or idea.
Maintainer	A person with merge rights who guides the project.
Contributor	Anyone who improves the project in any way.
Triage	Sorting incoming issues: reproducing, labelling, prioritising, closing duplicates.
Good first issue	A label for issues suitable for newcomers.
CI	Continuous integration: automatic tests, linting and builds on every change.
Lint	Automatic checks for code style and likely mistakes.
Rebase	Replaying your commits on top of a newer base so history stays linear.
Squash and merge	Combining a PR's commits into one when merging.
Draft PR	A PR marked as work in progress; not ready for full review.
LGTM	"Looks good to me": reviewer approval.
Nit	A tiny, optional suggestion in a review (naming, style).
CLA / DCO	Legal statements that you may submit your contribution.
Semantic versioning	MAJOR.MINOR.PATCH version numbers that signal the size of a change.
Changelog	A human-readable list of changes between releases.
Bus factor	How many people could disappear before a project stalls; low is risky.
Bikeshedding	Long debates about trivial details while the hard problems wait.
Stale	A bot label for issues or PRs with no activity; they may be closed automatically.

TIP **Your next step**

Open Chapter 9. Today: do Day 1. Tomorrow: Day 2. In two weeks you will have an open pull request, and that will change how you see every project you use.

Everyone you admire in open source once opened a first pull request, felt unsure, and clicked the button anyway.